

C++ Placement Prep — Day 1

C++ Fundamentals & Object-Oriented Programming Basics

Scope for today: C++ syntax differences from C, Classes & Objects, Access Specifiers, Constructors & Destructors, and the *this* pointer.

How to use this document: Read each concept, then try the programming examples yourself before checking the explanation. Attempt every practice question before looking at the answer key. Answers are printed after each section, upside-down in logic but right-side up on the page — just don't peek early.

Covers: theory notes • programming examples • MCQs • fill-in-the-blanks • predict-the-output • find-the-error • interview communication tips

1. C++ vs C — What's Actually New For You

You already know C well, so don't relearn from zero. Focus on this delta.

Feature	C	C++
Output	<code>printf("%d", x);</code>	<code>cout << x;</code>
Input	<code>scanf("%d", &x);</code>	<code>cin >> x;</code>
Boolean type	Not built-in (use int)	<code>bool (true / false)</code>
Dynamic memory	<code>malloc()</code> / <code>free()</code>	<code>new</code> / <code>delete</code>
Alias to a variable	Not available (only pointers)	Reference: <code>int &r = x;</code>
Same function, diff signature	Not allowed	Function overloading allowed
Default parameter values	Not allowed	<code>void f(int x=5)</code>
Comments	<code>/* ... */</code> only (C89)	<code>//</code> and <code>/* ... */</code>

References — the one C doesn't have at all

A reference is another name for an existing variable. Unlike a pointer, it cannot be null and cannot be reassigned to refer to something else after initialization.

```
int x = 10;
int &r = x;    // r IS x, not a pointer to x
r = 20;        // this changes x too
cout << x;     // prints 20

void increment(int &n) {    // pass by reference
    n = n + 1;
}
increment(x);    // x becomes 21, no & or * needed at call site
```

Interview angle: they may ask "*reference vs pointer*". Key points: reference must be initialized when declared, cannot be null, cannot be reseated, no separate dereference syntax needed.

2. Classes and Objects

A **class** is a blueprint. An **object** is an actual instance built from that blueprint. You already know this idea from a C **struct** — a class just goes further by bundling in functions (behavior) along with the data, and by controlling who can access what.

struct vs class — the only real difference

- In C++, **struct** members are **public** by default.
- In C++, **class** members are **private** by default.
- Both can have constructors, member functions, and access specifiers. Functionally interchangeable if you write the access specifiers explicitly — but convention is: struct for plain data, class for objects with behavior.

Basic class syntax

```
class Rectangle {
    private:
        double length;
        double width;

    public:
        void setDimensions(double l, double w) {
            length = l;
            width = w;
        }
        double getArea() {
            return length * width;
        }
};

int main() {
    Rectangle r;           // object created
    r.setDimensions(5.0, 3.0);
    cout << r.getArea();   // 15
}
```

Defining member functions outside the class (scope resolution ::)

Common in real projects: declare the function inside the class, define it outside using **ClassName::functionName**.

```
class Rectangle {
    public:
        double length, width;
        double getArea();    // declaration only
};

double Rectangle::getArea() { // definition outside
    return length * width;
}
```

Interviewers like asking why you'd do this: it separates interface (header/.h) from implementation (.cpp) — same reason you split .h and .c files in embedded C projects.

3. Access Specifiers

Specifier	Accessible from
private	Only within the same class
protected	Same class + derived (child) classes
public	Anywhere the object is visible

Why it matters (this is the actual interview point): access specifiers enforce **encapsulation** — hiding internal data so it can only be changed through controlled functions (getters/setters), instead of letting any code touch it directly. If asked "why not just make everything public?", answer with: prevents invalid states, hides implementation detail so it can change later without breaking other code, and makes debugging easier since data changes only happen in one place.

4. Constructors and Destructors

Constructor

A special member function, same name as the class, no return type, called automatically when an object is created. Used to initialize data members.

```
class Rectangle {
    public:
        double length, width;

        Rectangle() {                // default constructor
            length = 0; width = 0;
        }
        Rectangle(double l, double w) { // parameterized constructor
            length = l; width = w;
        }
};

Rectangle r1;                // calls default constructor
Rectangle r2(5.0, 3.0);     // calls parameterized constructor
```

This is **constructor overloading** — same idea as function overloading, applied to constructors. The compiler picks the right one based on the arguments you pass.

Initializer list (mention if asked about 'better' way to init)

```
Rectangle(double l, double w) : length(l), width(w) {
    // body can be empty here
}
```

Slightly more efficient than assigning inside the body, and required for initializing const members or reference members.

Destructor

Also a special function, name is **~ClassName()**, no return type, no parameters, called automatically when the object goes out of scope or is deleted. Used for cleanup (freeing memory, closing files).

```
class Rectangle {
    public:
        double *data;
        Rectangle() { data = new double[10]; }
        ~Rectangle() { delete[] data; } // cleanup here
};
```

Interview angle: *"Can a class have more than one destructor?"* No — unlike constructors, destructors cannot be overloaded, since they take no arguments at all.

5. The 'this' Pointer

Inside any non-static member function, **this** is a hidden pointer to the object the function was called on. Most common real use: resolving a naming conflict between a parameter and a member variable with the same name.

```
class Rectangle {
    public:
        double length, width;

        void setLength(double length) {    // parameter shadows member
            this->length = length;         // this->length = the member
        }                                  // length = the parameter
};
```

Second common use: returning the object itself from a member function to allow method chaining (e.g. `obj.setA(1).setB(2);`) — worth mentioning if the interviewer pushes further, but the naming-conflict case above is the one to have ready.

6. Worked Programming Examples

Example 1 — Put it all together: a Rectangle class

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length, width;
public:
    Rectangle() : length(0), width(0) {}
    Rectangle(double l, double w) : length(l), width(w) {}

    void setDimensions(double l, double w) {
        length = l;
        width = w;
    }
    double getArea() { return length * width; }
    double getPerimeter() { return 2 * (length + width); }
    ~Rectangle() { cout << "Rectangle destroyed" << endl; }
};

int main() {
    Rectangle r1;                // default constructor
    Rectangle r2(4.0, 6.0);      // parameterized constructor

    r1.setDimensions(2.0, 3.0);

    cout << "r1 area: " << r1.getArea() << endl;    // 6
    cout << "r2 perimeter: " << r2.getPerimeter() << endl; // 20
    return 0;
}
```

Example 2 — this pointer in action

```
class Counter {
private:
    int count;
public:
    Counter(int count) { this->count = count; }
    Counter& increment() {
        this->count++;
        return *this;    // enables chaining
    }
    int getCount() { return count; }
};

int main() {
    Counter c(5);
    c.increment().increment().increment();    // chained calls
    cout << c.getCount();                      // 8
}
```

Try this yourself before moving on

Write a class **BankAccount** with a private double **balance**, a constructor that takes an initial balance, a **deposit(double amt)** function, a **withdraw(double amt)** function that refuses to let balance go negative, and a **getBalance()** function. Add a destructor that prints "Account closed". This is a realistic

MCQ/coding-round style question — actually write it out by hand before checking your own logic.

7. Multiple Choice Questions

Q1. What is the default access specifier for members of a class (not struct) in C++?

- A) public
- B) private
- C) protected
- D) none, must always be specified

Q2. Which of these is called automatically when an object goes out of scope?

- A) Constructor
- B) Copy constructor
- C) Destructor
- D) this pointer

Q3. Can constructors be overloaded in C++?

- A) Yes, based on number/type of parameters
- B) No, never
- C) Only if the class has no destructor
- D) Only for structs

Q4. What does the following declare? `int &r; = x;`

- A) A pointer to x
- B) A reference to x, an alias for x
- C) A copy of x
- D) Invalid syntax in C++

Q5. Which statement about destructors is TRUE?

- A) They can take arguments
- B) They can be overloaded
- C) They have no return type and no parameters
- D) They must be called manually always

Q6. Inside a member function, what does 'this' represent?

- A) The class name
- B) A pointer to the current object
- C) A reference to the constructor
- D) A static variable

Q7. By default, members of a struct in C++ are:

- A) private
- B) protected
- C) public
- D) undefined behavior

Q8. Which is NOT a valid reason to use private access specifiers?

- A) Encapsulation
- B) Preventing invalid direct modification of data

C) Making the program run faster at compile time

D) Hiding implementation detail from other code

Answer key: Q1-B, Q2-C, Q3-A, Q4-B, Q5-C, Q6-B, Q7-C, Q8-C

8. Fill in the Blanks

1. The _____ operator is used to define a member function outside the class body.
2. A class member function with the same name as the class and no return type is called a _____.
3. The destructor of a class named 'Car' is written as _____.
4. Inside a member function, _____ points to the object on which the function was called.
5. By default, members of a _____ are private, while members of a _____ are public.

Answers: 1. scope resolution (::) 2. constructor 3. ~Car() 4. this 5. class, struct

9. Predict the Output

Q1

```
class Demo {
public:
    int x;
    Demo(int x) { this->x = x; }
    void show() { cout << x << " "; }
};

int main() {
    Demo d1(10), d2(20);
    d1.show();
    d2.show();
}
```

Answer: 10 20 — each object keeps its own separate copy of x, this->x correctly assigns the parameter to that object's member.

Q2

```
class Box {
public:
    Box() { cout << "constructed "; }
    ~Box() { cout << "destroyed "; }
};

int main() {
    Box b;
    cout << "in main ";
}
```

Answer: constructed in main destroyed — constructor runs at creation, destructor runs automatically when b goes out of scope at the end of main.

Q3

```
class Point {
private:
    int x;
public:
    Point(int val) : x(val) {}
    int getX() { return x; }
};

int main() {
    Point p(7);
    cout << p.x;    // line in question
}
```

Answer: Compile error — x is private, cannot be accessed directly as p.x from outside the class. Must use p.getX().

10. Find the Error

Q1

```
class Circle {
    public:
        double radius;
        Circle(double r) { radius = r; }
        Circle() { }    // ERROR here – spot it
};

int main() {
    Circle c1(5.0);
    Circle c2;
    c2.radius = 3.0;
}
```

Answer: No error actually — this is valid! Two constructors with different parameter lists is legal overloading. (This one is a trap to check you don't over-flag valid overloading as an error.)

Q2

```
class Employee {
    private:
        string name;
    public:
        Employee(string n) { name = n; }
};

int main() {
    Employee e;    // ERROR
}
```

Answer: No matching default constructor. Once you define ANY parameterized constructor, the compiler stops auto-generating a default constructor. `Employee e;` fails to compile unless you also define `Employee()`.

Q3

```
class Test {
    public:
        ~Test(int x) { }    // ERROR
};
```

Answer: Destructors cannot take parameters, and cannot be overloaded. `~Test(int x)` is invalid — must be exactly `~Test()`.

11. What Interviewers Are Really Checking Here

- Can you explain **why** OOP concepts exist, not just recite definitions — e.g. why encapsulation matters, not just "private hides data".
- Can you trace constructor/destructor call order without running the code in your head slowly — practice this with the predict-the-output questions above until it's fast.
- Do you understand that C++ classes are a natural extension of C structs, not something totally alien — connecting it back to something you know builds credibility fast.
- Do you know the DEFAULT behavior the compiler gives you for free (default constructor, when it disappears) — this is a very common trick question (see Find-the-Error Q2 above).
- Can you write a small class from scratch on a whiteboard/editor without syntax hesitation — practice the BankAccount exercise from Section 6 until it's automatic.

12. How to Answer & Communicate This Topic Well

Structure every answer the same way

- **1. Definition** — one clean sentence.
- **2. Why it matters** — the real-world reason it exists, not just what it does.
- **3. Small example** — a short code snippet or a spoken mini-example.
- **4. Connect it** — link it to something you already know (C structs, embedded projects) if it naturally fits. Don't force it.

Example spoken answer for "What is encapsulation?":

"Encapsulation means bundling data and the functions that operate on that data together, and restricting direct access to the data using access specifiers like `private`. It matters because it stops other parts of the program from putting an object into an invalid state, and it lets you change the internal implementation later without breaking code that uses the class. For example, in a `BankAccount` class I'd keep `'balance'` `private` and only allow changes through a `withdraw()` function that checks for sufficient funds first."

If you don't know the answer

- Never go silent. Say what you DO know first: "I haven't used that exact feature, but based on how constructors work, I'd expect..."
- It's fine to think out loud — interviewers are often evaluating your reasoning process, not just the final answer.
- If genuinely stuck, ask one clarifying question rather than guessing wildly — shows structured thinking under pressure.

Practical delivery tips

- Speak slightly slower than feels natural — nerves speed people up and make answers sound rushed/unsure even when the content is correct.
- Use the word "because" often — it forces you to justify rather than just state facts.
- For coding-round explanations: narrate your plan before typing ("I'll use a private member for the count, and a public increment function...") — this alone signals OOP thinking clearly.
- It is completely fine to say "give me a moment to think" before answering a conceptual question — silence is better than a rushed wrong answer.

End of Day 1 material. Attempt every practice section before checking answers — that's how it actually sticks in a week-long crash prep. Tomorrow: Inheritance & Polymorphism.